Gerenciamento de Configuração de Software

Prof. Daniel Callegari, 2026/1

Trabalho 1 – Prática com Git em Times

O trabalho consiste em observar os requisitos do trabalho, implementar uma solução seguindo os conceitos vistos em aula e produzir um relatório final.

O trabalho deverá ser realizado em times compostos por 9 membros. Cópias ou tentativas de fraude resultarão em nota zero para todos os envolvidos.

Data de entrega: 15/05/2020

Atividades

Cada time deverá:

1. Cada grupo deve criar um único repositório no GitHub. O repositório deve ser público e seu endereço deve ser copiado para o relatório.
2. Definir o seu processo de *branching* (fluxo de trabalho), com base em um dos fluxos estudados.
3. Descrever as regras do fluxo de trabalho no relatório e segui-las ao longo da implementação.
    a. O professor verificará a participação de cada membro do grupo através do log do git no repositório do time. Cada aluno deverá participar de pelo menos duas *features* ou correções de *bug*, com uma contribuição substancial.
    b. As submissões ao repositório no GitHub deverão ocorrer via *Pull Requests*.
    c. Os movimentos no repositório deverão acontecer até no máximo a data de entrega. Serão desconsiderados os trabalhos que tiverem movimentos depois da data de entrega.
4. Escrever o software a partir do repositório inicial criado de acordo com os requisitos e com o fluxo de trabalho definido pelo time.
5. Entregar um relatório via Moodle até a data limite. O relatório deverá conter:
    a. Folha de rosto com nomes completos dos componentes do time e seus nomes de usuário no GitHub.
    b. Link para o repositório público no GitHub.
    c. Descrição clara do fluxo de trabalho adotado (pode ser uma variação dos fluxos vistos em aula, adaptado pelo time).
    d. Demonstração de que o time seguiu o fluxo definido. Deve conter capturas de tela e descrições que deixem isto claro. (sugere-se capturar o Network graph do GitHub: Insights/Network)
    e. Tabela com os nomes dos componentes do grupo e suas contribuições (ex. links para commits ou pull-requests registrados no GitHub).
    f. Conclusão do grupo contendo reflexões sobre as dificuldades encontradas, como foram superadas e quais as lições aprendidas. A conclusão deverá conter uma nota de autoavaliação do grupo (0,0 a 10,0). A conclusão deverá conter uma nota de autoavaliação do grupo (0,0 a 10,0). Também deverá listar e comentar claramente a contribuição de cada aluno no trabalho.

Critérios de Avaliação

Avaliação: 30% da nota é resultado do individual e 70% da nota advém do resultado do grupo. Cada aluno deverá garantir que seu nome/usuário conste (em bom número de ocorrências) nos *commits* do histórico do git, confirmando a sua participação no processo de desenvolvimento, seguindo o fluxo de trabalho definido pelo grupo.

Dica: utilize o comando abaixo para uma métrica básica por usuário:

```
git shortlog -s -n --all --no-merges
```

Item	**Pontos**
Descrição clara do fluxo de trabalho adotado pelo grupo -- nomes de branches, regras de uso, comandos utilizados, exemplos, etc.	3,0
Demonstração de que o grupo seguiu o fluxo de trabalho definido – capturas de tela, descrições, anotações sobre trechos de código, log do git, etc.	4,0
Qualidade geral do relatório, em atendimento os itens listados no item Atividades.	2,0
Conclusão com autoavaliação do grupo.	1,0
TOTAL	**10,0**

Requisitos Gerais

1. Deseja-se um **sistema de autorização de exames médicos**.
2. O sistema deverá ser implementado em **Java Console** /ou/ **Web Puro** (apenas HTML+CSS+Javascript). Não utilizar frameworks ou outras dependências. A ideia é ter a base de código o mais simples possível. Cuidar para que **todos** os membros do time conheçam a(s) linguagem(s) escolhida(s).
3. Não implementar um mecanismo de persistência de dados. O sistema deverá manter dados apenas em memória durante a execução.
4. Não implementar um mecanismo de login. No entanto, deverá ser possível identificar/alterar o usuário/operador que está usando o sistema no momento. Para cada operador, deve-se saber o nome e as suas iniciais.
5. O sistema deverá iniciar com dados já preenchidos (em bom número e de boa qualidade), de forma a facilitar os testes.

Detalhamento

1) O sistema deverá permitir escolher o usuário atual a qualquer momento. Cada usuário tem um identificador, um nome e o seu tipo: médico, paciente ou administrador.
2) O sistema deverá permitir a um médico incluir uma nova autorização de exame (não é necessário implementar edição ou exclusão). Cada autorização de exame tem um código numérico sequencial iniciado em 1000, uma data de cadastro, o médico solicitante, o paciente e o exame solicitado (ex. RaioX, Tomografia etc). Cada autorização permite apenas 1 exame.
   a) Todos os campos devem ser preenchidos via seleção (textual ou visual) e não via digitação de texto puro (ou seja, tecnicamente devem se referir aos objetos em memória e não ser apenas texto puro).
   b) A lista de exames é fixa e deve conter pelo menos 10 tipos diferentes.
3) O sistema deverá permitir a um médico listar todas as autorizações de exame filtrando por paciente ou por tipo de exame. Listar sempre ordenado pela data.
4) O sistema deverá permitir a um paciente marcar um exame como realizado, adicionando a data em que foi realizado.
   a) O sistema não deve permitir essa operação se a data informada for anterior à da solicitação ou for posterior a 30 dias da solicitação.
5) O sistema deverá permitir a um paciente listar as suas autorizações de exame, ordenadas pela data.
6) O sistema deverá permitir a um administrador incluir um novo usuário (médico, paciente ou outro administrador).
7) O sistema deverá permitir ao administrador buscar um médico ou paciente por parte de seu nome e listar as autorizações daquele usuário.
8) O sistema deverá permitir ao administrador ver estatísticas gerais contendo:
   a) Número de médicos ###
   b) Número de pacientes ###
   c) Número de autorizações emitidas
   d) Percentual de autorizações com exames já realizados
9) [Duas funcionalidades a mais à escolha do grupo]

Bom trabalho!